

ENGINEERING ENABLEMENT

Code Review Harness

Agentic System Curriculum

Instructor guide, learner labs, prompt-engineering standard, and 55-agent drafting plan

Core philosophy Evidence is observable. Assessment is bounded. Recommendations inform. Humans decide.

Repository-aligned edition

Based on the Agent Catalog, universal and layer contracts, workflow model, and governance guidance. Prepared for young engineers learning to design, operate, and test contract-driven agents.

How to use this curriculum

This is a twelve-session progression from first principles to a production-style prompt package. The course treats prompts as governed software artifacts: they inherit contracts, operate inside a stateful harness, produce typed artifacts, preserve evidence, and stop at the boundary of human authority.

Recommended delivery Two sessions per week for six weeks, or one session per week for a twelve-week academy. Sessions 3–11 are designed for 120 minutes; the capstone uses 180 minutes.

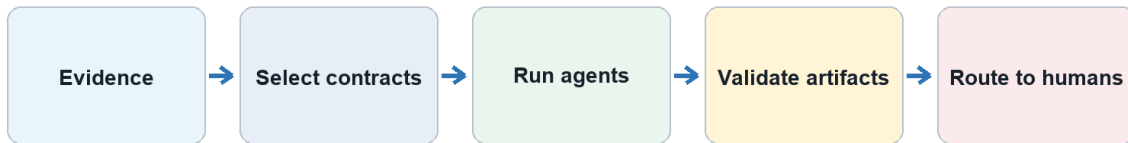
Course-level outcomes

- Explain why the harness—not the model—is the trust boundary for agentic review.
- Read the catalog and route work to the correct agent, contract, and consumer.
- Design stateful workflows with explicit guards, failure paths, and human decision points.
- Draft role prompts that implement inherited contracts without scope or authority leakage.
- Test prompts against adversarial cases and evolve them through controlled evidence.
- Produce reviewed prompt packages for every agent in the catalog.

Course map

Phase	Sessions	Emphasis	Learner artifact
Foundations	1–3	Harness philosophy and evidence	Evidence-flow map
Architecture	4–7	Contracts, identities, states, orchestration	Validated workflow design
Prompt engineering	8–11	Drafting, specialization, red-teaming, evaluation	Tested prompt candidate
Capstone	12	Factory-scale authoring and review	Operational prompt package

The harness turns review into a controlled evidence flow



Control plane: identity • scope • provenance • confidence • authority • integrity

Figure 1. The harness controls evidence, execution, validation, and routing.

Facilitation schema used in every session

1. Open with a concrete failure or decision question.
2. Teach one mental model and tie it to repository artifacts.
3. Walk a worked example using canonical agent and contract language.
4. Run a guided lab that creates an inspectable artifact.
5. Draft or improve a prompt component.
6. Evaluate with a contract-derived exit ticket.

Assessment philosophy

Score observable behavior, not eloquence. Learners earn credit when their artifacts are grounded, complete, bounded, reproducible, calibrated, consumer-ready, and explicit about human authority. Every session contributes evidence to the capstone.

Repository basis: README.md; docs/philosophy.md; agents/agent-registry.md; contracts/contract-catalog.md

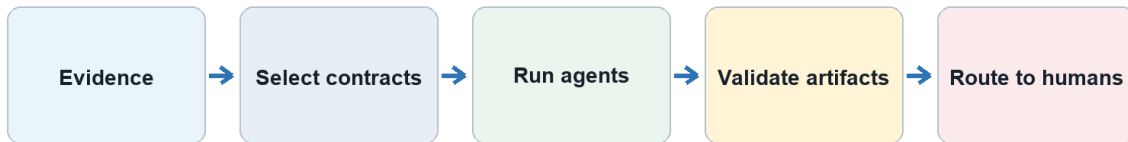
Session 1: Why a Harness Exists

Duration: 90 minutes | **Driving question:** Why is a strong model still not a trustworthy review system?

Learning outcomes

- Distinguish an AI model, an agent, a workflow, and a harness.
- Explain how the harness constrains scope, evidence, outputs, and authority.
- Trace one review request from evidence intake to human decision.

The harness turns review into a controlled evidence flow



Control plane: identity • scope • provenance • confidence • authority • integrity

Session visual: Why a Harness Exists.

Core mental model

Concept	Working definition
Model	Generates candidate reasoning and language.
Agent	A model operating under a role, contract, tools, and bounded context.
Workflow	A sequence of states and transitions among agents and artifacts.
Harness	The control plane that schedules, validates, records, and routes the work.

Facilitation run of show

1. Hook: compare a clever answer with an auditable review.
2. Teach the harness flow.
3. Walk a secrets-review example.
4. Lab: mark control points.
5. Exit ticket.

Instructor narrative

Open by asking: “Why is a strong model still not a trustworthy review system?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Give teams a repository change and an unconstrained review prompt. Ask them to identify every place where the result could become untraceable, over-broad, or falsely authoritative. Teams add one harness control for each failure.

Prompt engineering lab

Drafting task Rewrite “Review this code” as a bounded request that declares evidence, scope, consumer, output contract, and human decision authority.

Exit ticket and evidence of learning

Learner can name at least four controls provided by the harness and explain why the model alone cannot provide them.

Watch for these misconceptions

- A harness is only a prompt wrapper.

- More autonomous agents always mean better review.
- A fluent result is equivalent to a validated artifact.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: docs/philosophy.md; orchestration/workflow-model.md; contracts/universal-agent-contract.md

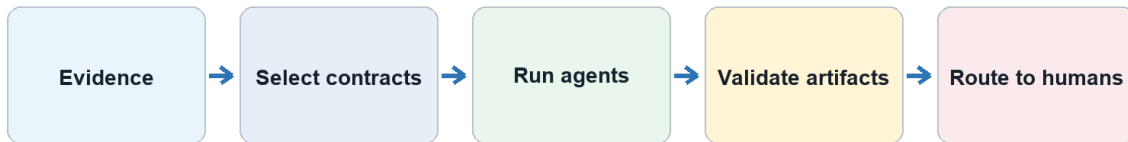
Session 2: Philosophy: Evidence First, Humans Decide

Duration: 90 minutes | **Driving question:** What should an agent be allowed to claim?

Learning outcomes

- Apply the evidence-first and human-authority principles.
- Separate observation, assessment, recommendation, and decision.
- Recognize persuasive language that exceeds the evidence.

The harness turns review into a controlled evidence flow



Control plane: identity • scope • provenance • confidence • authority • integrity

Session visual: Philosophy: Evidence First, Humans Decide.

Core mental model

Concept	Working definition
Evidence	An immutable observable source with provenance.
Assessment	A bounded interpretation under declared criteria.
Recommendation	An option for a human authority to consider.
Decision	A human-owned act that changes acceptance, priority, or risk posture.

Facilitation run of show

1. Read the philosophy.
2. Sort statements by epistemic type.
3. Red-team overclaiming.
4. Rewrite with calibrated authority.
5. Debrief.

Instructor narrative

Open by asking: “What should an agent be allowed to claim?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Teams classify ten statements as evidence, assessment, recommendation, or decision. They must defend each classification and identify the evidence needed to strengthen weak statements.

Prompt engineering lab

Drafting task Add authority language that forces the agent to request decisions instead of granting approvals, accepting risk, or declaring compliance.

Exit ticket and evidence of learning

Learner rewrites an overconfident conclusion into an evidence-linked assessment with a human decision request.

Watch for these misconceptions

- Human-in-the-loop means a person merely clicks approve.

- Confidence is a substitute for evidence.
- A recommendation may silently become a decision.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: docs/philosophy.md; governance/human-review-and-maturity.md; contracts/evidence-contract.md

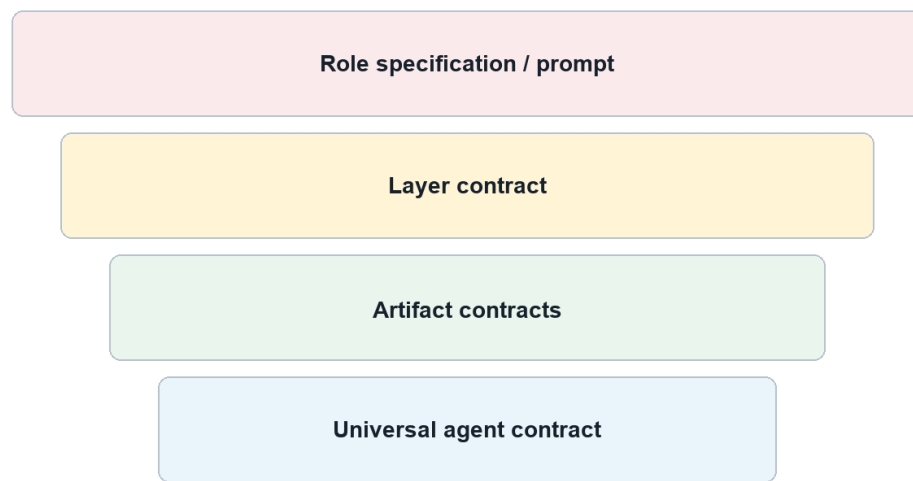
Session 3: Evidence, Provenance, and Traceability

Duration: 120 minutes | **Driving question:** Can another engineer reproduce why this finding exists?

Learning outcomes

- Build an evidence chain from source to finding to decision request.
- Use stable identifiers, provenance, coverage, and confidence correctly.
- Identify missing, stale, conflicting, or inadmissible evidence.

Contracts compose from universal rules to role-specific behavior



More specific layers add constraints; they do not erase inherited obligations.

Session visual: Evidence, Provenance, and Traceability.

Core mental model

Concept	Working definition
Provenance	Where the artifact came from and how it was obtained.
Coverage	What was inspected, what was not, and why.
Traceability	Stable links among inputs, reasoning products, and consumers.

Concept	Working definition
Confidence	Calibrated support for a claim—not certainty theater.

Facilitation run of show

1. Evidence contract.
2. Traceability graph.
3. Coverage exercise.
4. Conflict handling.
5. Mini-review.

Instructor narrative

Open by asking: “Can another engineer reproduce why this finding exists?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Build a traceability chain for a vulnerable dependency: lockfile line → evidence item → finding → CAPA → consumer → decision request. Then remove one link and explain the failure.

Prompt engineering lab

Drafting task Write evidence admission rules and citation requirements that prevent unsupported findings and require explicit incomplete-input behavior.

Exit ticket and evidence of learning

Learner produces a complete, reproducible evidence chain and names any coverage limits.

Watch for these misconceptions

- A URL alone is provenance.
- High confidence repairs missing evidence.
- Coverage can be inferred from the length of a report.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: contracts/evidence-contract.md; contracts/evidence-flow-model.md;
contracts/universal-agent-contract.md

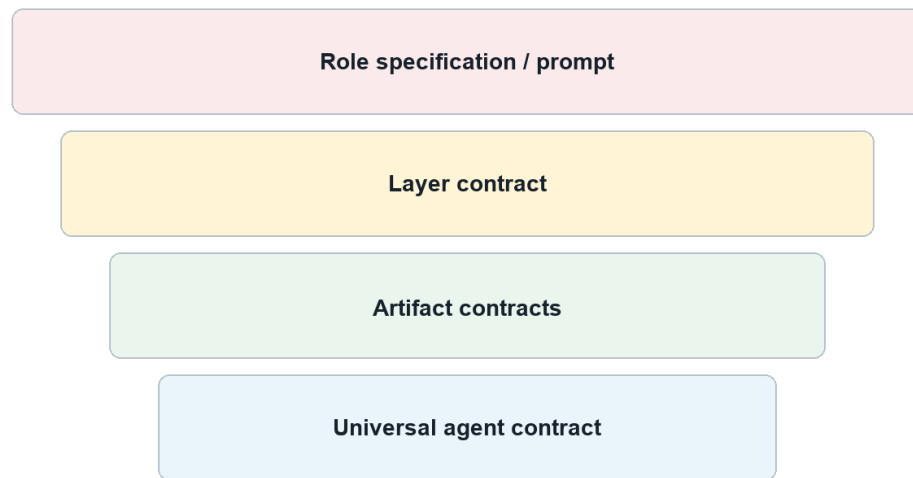
Session 4: How Contracts Work

Duration: 120 minutes | **Driving question:** How do independent agents exchange trustworthy artifacts?

Learning outcomes

- Explain contract inheritance and additive extension.
- Map producers and consumers across contract classes.
- Validate a sample artifact against the universal envelope.

Contracts compose from universal rules to role-specific behavior



More specific layers add constraints; they do not erase inherited obligations.

Session visual: How Contracts Work.

Core mental model

Concept	Working definition
Universal contract	Identity, artifact, execution, scope, provenance, confidence, authority, and integrity.
Layer contract	Rules shared by specialist, product, capability, enterprise, work, or orchestration roles.

Concept	Working definition
Artifact contract	Schema and semantics for evidence, findings/CAPA, patterns, and insights.
Role specification	The authoritative question, scope, method, inputs, outputs, and consumers for one agent.

Facilitation run of show

1. Contract stack.
2. Envelope field tour.
3. Producer/consumer mapping.
4. Validation lab.
5. Versioning discussion.

Instructor narrative

Open by asking: “How do independent agents exchange trustworthy artifacts?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Validate a deliberately flawed agent output. Mark missing envelope fields, semantic violations, unauthorized decisions, and broken consumer links. Repair it without changing the evidence.

Prompt engineering lab

Drafting task Translate five universal contract obligations into explicit prompt clauses and corresponding output fields.

Exit ticket and evidence of learning

Learner explains why a role prompt cannot override the universal contract and identifies when a major contract version is required.

Watch for these misconceptions

- A schema alone is a contract.
- A child contract may remove inconvenient parent rules.
- Changing field meaning is a harmless minor version.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: contracts/contract-catalog.md; contracts/contract-dependency-graph.md; contracts/style-and-validation.md

Session 5: Agent Identity and Architecture Layers

Duration: 90 minutes | **Driving question:** Which agent should answer which question?

Learning outcomes

- Navigate the 55-agent catalog and its six layers.
- Use canonical designation, UUID, version, status, and specification.
- Route a question to the narrowest authoritative agent.

The catalog separates observation, synthesis, and coordination

Enterprise	12 agents • system-of-systems decision support
Capability	11 agents • cross-product mission and readiness
Product	4 agents • product-level correlation
Specialist	22 agents • bounded domain review
Work + orchestration	6 agents • reusable processing and routing

Session visual: Agent Identity and Architecture Layers.

Core mental model

Concept	Working definition
Identity	Immutable UUID plus canonical designation and version.
Layer	The level of evidence aggregation and authoritative question.
Status	Baseline, seed, or planned readiness for scheduling.

Concept	Working definition
Boundary	A role's owned question and explicit exclusions.

Facilitation run of show

1. Catalog tour.
2. Architecture layers.
3. Identity rules.
4. Routing game.
5. Boundary critique.

Instructor narrative

Open by asking: “Which agent should answer which question?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Teams receive twelve questions and route each to a catalog agent. For every choice they cite the authoritative question, layer, and status; they also name one tempting but incorrect agent.

Prompt engineering lab

Drafting task Draft the identity and boundary blocks for one specialist agent and one synthesis agent. Ensure neither duplicates the other.

Exit ticket and evidence of learning

Learner chooses the correct agent for a scenario and explains the layer boundary in one sentence.

Watch for these misconceptions

- Display names are stable identifiers.
- Synthesis agents re-run all specialist reviews.
- Planned agents are production-ready because they appear in the registry.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: agents/agent-registry.md; agents/agent-identities.json; agents/agent-naming-and-identity-standard.md

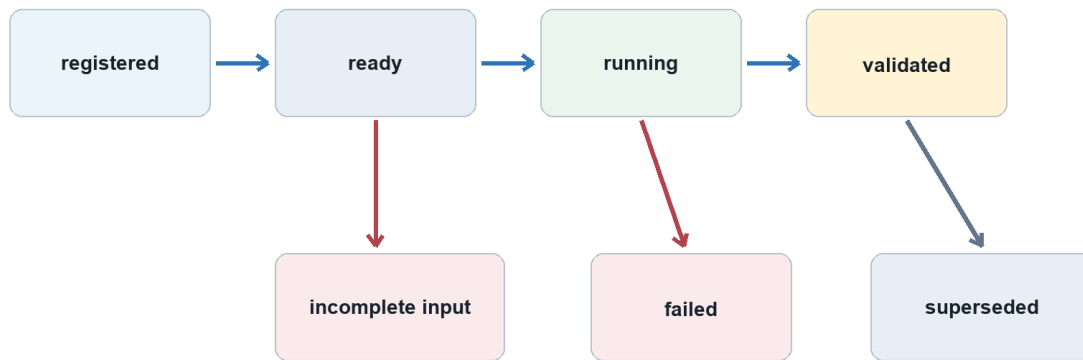
Session 6: State Machines: Making Work Observable

Duration: 120 minutes | **Driving question:** What exactly must be true before work advances?

Learning outcomes

- Model agent execution as states, events, transitions, and guards.
- Distinguish incomplete input, failure, and supersession.
- Design transitions that are deterministic and auditable.

A state machine makes progress, failure, and authority explicit



Guards answer “may this transition happen?”; events explain “why did it happen?”

Session visual: State Machines: Making Work Observable.

Core mental model

Concept	Working definition
State	A durable description of where an execution or artifact is.
Event	The occurrence that asks the workflow to transition.
Guard	A predicate that must be true for the transition to occur.

Concept	Working definition
Terminal state	A completed, failed, incomplete, or superseded outcome.

Facilitation run of show

1. Human example.
2. Harness state model.
3. Guard design.
4. Failure branches.
5. Tabletop simulation.

Instructor narrative

Open by asking: “What exactly must be true before work advances?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Run a tabletop execution. Learners draw event cards and may transition only when guards pass. They must log the event, state change, artifact IDs, and failure reason.

Prompt engineering lab

Drafting task Write prompt behavior for missing prerequisites: do not guess, emit the incomplete-input state, list missing evidence, and identify the safe next action.

Exit ticket and evidence of learning

Learner can explain the difference between an event and a guard and can model at least two failure paths.

Watch for these misconceptions

- State is the same as a progress message.
- Retrying erases the failed attempt.
- All errors should transition to failed.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: orchestration/workflow-model.md; contracts/universal-agent-contract.md

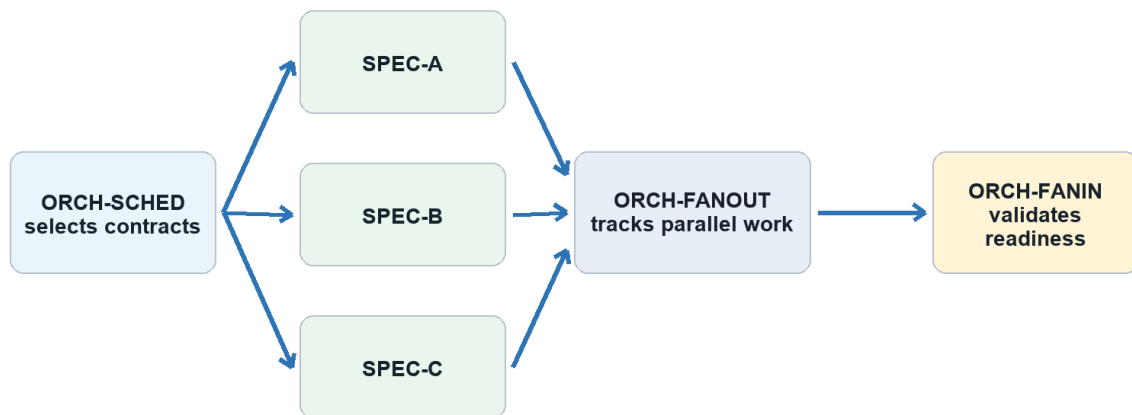
Session 7: Orchestration: Fan-Out, Fan-In, and Gates

Duration: 120 minutes | **Driving question:** How do we coordinate many agents without hiding causality?

Learning outcomes

- Explain scheduler, fan-out, fan-in, and evidence-gate responsibilities.
- Identify safe parallelism and required dependencies.
- Prevent orchestrators and synthesizers from becoming unauthorized reviewers.

Orchestration coordinates work without becoming a reviewer



Schedulers route. Reviewers assess. Humans decide.

Session visual: Orchestration: Fan-Out, Fan-In, and Gates.

Core mental model

Concept	Working definition
ORCH-SCHED	Selects registered roles and contract versions.
ORCH-FANOUT	Dispatches independent work and tracks executions.
ORCH-FANIN	Validates prerequisites and assembles consumable input sets.

Concept	Working definition
ENT-EVIDENCE	Gates enterprise input fitness before enterprise analysis.

Facilitation run of show

1. Dependency graph.
2. Parallelism rules.
3. Fan-in validation.
4. Enterprise gate.
5. Workflow design lab.

Instructor narrative

Open by asking: “How do we coordinate many agents without hiding causality?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Design a workflow for a release review. Mark dependencies, parallel branches, join criteria, retries, timeouts, and the exact point where a human decision is requested.

Prompt engineering lab

Drafting task Draft an orchestration prompt that schedules and validates but never generates domain findings.

Exit ticket and evidence of learning

Learner produces a dependency graph with safe parallelism and an explicit join guard.

Watch for these misconceptions

- The orchestrator is the smartest reviewer.
- Fan-in means concatenate every report.
- Enterprise synthesis may overrule the evidence gate.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: orchestration/workflow-model.md; contracts/orchestration-agent-contract.md; agents/enterprise/evidence-validation-gate.md

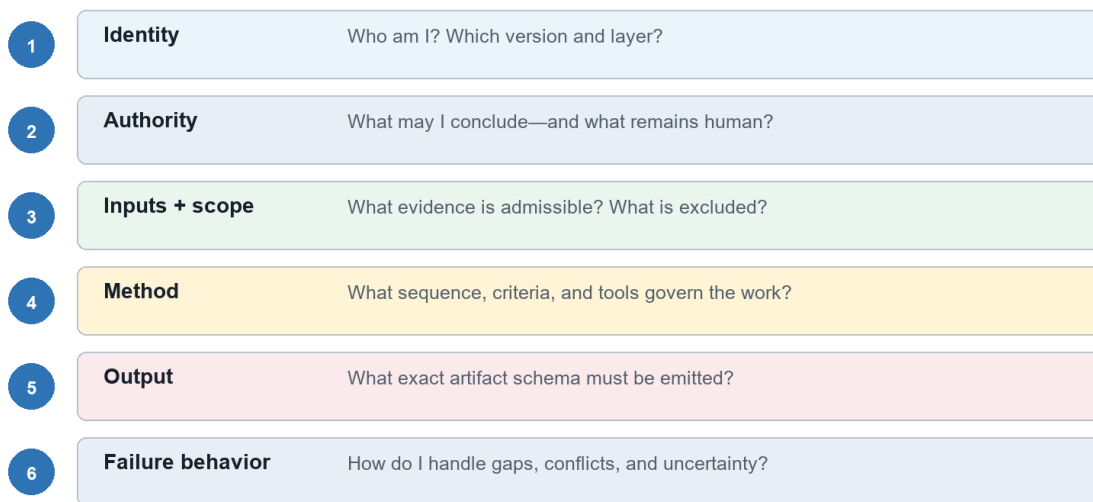
Session 8: The Prompt Is an Implementation of the Contract

Duration: 120 minutes | **Driving question:** How do we turn a specification into reliable model behavior?

Learning outcomes

- Use the platform's universal prompt structure.
- Trace every prompt clause to a contract obligation or role requirement.
- Keep authority, evidence, method, and output mutually consistent.

A production prompt is an executable projection of the contract



Session visual: The Prompt Is an Implementation of the Contract.

Core mental model

Concept	Working definition
System rules	Non-negotiable platform and safety constraints.
Role block	Identity, mission, authoritative question, and boundaries.
Execution block	Inputs, evidence rules, method, tools, and state behavior.

Concept	Working definition
Output block	Exact schema, confidence, conflicts, consumers, and decision requests.

Facilitation run of show

1. Prompt anatomy.
2. Contract-to-prompt mapping.
3. Worked example.
4. Pair drafting.
5. Peer review.

Instructor narrative

Open by asking: “How do we turn a specification into reliable model behavior?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Convert a role specification into a prompt. Partners use a trace matrix to verify every instruction has an origin and every contract obligation has an implementation.

Prompt engineering lab

Drafting task Draft a complete first-pass prompt for a baseline specialist agent using the universal template.

Exit ticket and evidence of learning

Learner can defend each prompt section by pointing to the governing contract or role specification.

Watch for these misconceptions

- More detail always makes a prompt better.
- Persona language can replace a method.
- Output examples may contradict the schema if they read well.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: contracts/universal-agent-contract.md; contracts/specialist-agent-contract.md; agents/specialists/*.md

Session 9: Prompt Decomposition and Specialization

Duration: 120 minutes | **Driving question:** How do we keep 55 prompts coherent without making them identical?

Learning outcomes

- Separate shared prompt modules from role-specific instructions.
- Define authoritative questions, exclusions, consumers, and escalation paths.
- Design synthesis prompts that preserve provenance and disagreement.

The catalog separates observation, synthesis, and coordination

Enterprise	12 agents • system-of-systems decision support
Capability	11 agents • cross-product mission and readiness
Product	4 agents • product-level correlation
Specialist	22 agents • bounded domain review
Work + orchestration	6 agents • reusable processing and routing

Session visual: Prompt Decomposition and Specialization.

Core mental model

Concept	Working definition
Shared module	Reusable language for evidence, authority, confidence, and output integrity.
Specialization	Domain criteria, method, tools, and owned question.

Concept	Working definition
Synthesis	Correlation of validated artifacts without fabricating agreement.
Consumer fit	The output detail and semantics needed by the next contract.

Facilitation run of show

1. Prompt module map.
2. Specialist vs synthesizer.
3. Boundary patterns.
4. Drafting sprint.
5. Cross-team review.

Instructor narrative

Open by asking: “How do we keep 55 prompts coherent without making them identical?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Teams draft prompts for one agent from each layer, then compare repeated clauses. They extract stable shared modules and retain only genuine role specialization.

Prompt engineering lab

Drafting task Write a synthesis clause that preserves source IDs, conflicting assessments, coverage differences, and human decision authority.

Exit ticket and evidence of learning

Learner can identify duplicated policy language and move it to a shared module without erasing role boundaries.

Watch for these misconceptions

- Consistency means every prompt uses the same method.
- Synthesis should hide disagreement.
- A broad prompt is more reusable than a bounded prompt.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: agents/hierarchical-agent-architecture.md; contracts/contract-dependency-graph.md; contracts/enterprise-synthesis-contract.md

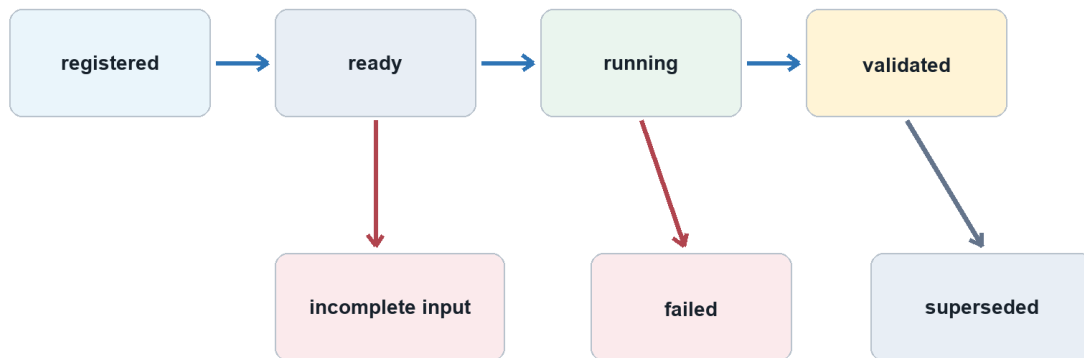
Session 10: Prompt Failure Modes and Adversarial Testing

Duration: 120 minutes | **Driving question:** How will this prompt fail under pressure?

Learning outcomes

- Recognize ambiguity, scope drift, evidence fabrication, authority leakage, and schema failure.
- Design adversarial cases for missing, conflicting, malicious, and excessive inputs.
- Specify safe failure behavior instead of optimistic guessing.

A state machine makes progress, failure, and authority explicit



Guards answer “may this transition happen?”; events explain “why did it happen?”

Session visual: Prompt Failure Modes and Adversarial Testing.

Core mental model

Concept	Working definition
Scope drift	The agent answers adjacent questions it does not own.
Authority leakage	Recommendations or scores become approvals or decisions.
Evidence laundering	Unsupported claims appear credible after synthesis.

Concept	Working definition
Prompt injection	Input content attempts to override the governing instruction hierarchy.

Facilitation run of show

1. Failure taxonomy.
2. Bad prompt teardown.
3. Adversarial case design.
4. Red-team exchange.
5. Repair sprint.

Instructor narrative

Open by asking: “How will this prompt fail under pressure?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Attack another team’s prompt with eight cases: missing inputs, contradictory evidence, stale evidence, malicious repository text, huge context, tool failure, schema conflict, and pressure to approve.

Prompt engineering lab

Drafting task Add explicit behavior for each attack without turning the prompt into an unreadable exception list.

Exit ticket and evidence of learning

Learner demonstrates that the repaired prompt fails closed, preserves provenance, and requests human action when appropriate.

Watch for these misconceptions

- Prompt injection is only a web-browser problem.
- Refusal is the only safe failure behavior.
- A JSON schema prevents semantic overreach.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: contracts/style-and-validation.md; contracts/sandboxed-contract-evolution.md; governance/integration-boundaries.md

Session 11: Evaluation, Calibration, and Prompt Evolution

Duration: 120 minutes | **Driving question:** How do we know a prompt is improving rather than merely changing?

Learning outcomes

- Build a contract-derived evaluation rubric and test corpus.
- Measure completeness, grounding, boundary adherence, calibration, and usefulness.
- Evolve prompts and contracts through controlled, auditable change.

A production prompt is an executable projection of the contract

1	Identity	Who am I? Which version and layer?
2	Authority	What may I conclude—and what remains human?
3	Inputs + scope	What evidence is admissible? What is excluded?
4	Method	What sequence, criteria, and tools govern the work?
5	Output	What exact artifact schema must be emitted?
6	Failure behavior	How do I handle gaps, conflicts, and uncertainty?

Session visual: Evaluation, Calibration, and Prompt Evolution.

Core mental model

Concept	Working definition
Evaluation case	Inputs plus expected invariants and failure behavior.
Calibration	Confidence and severity correspond to evidence strength and impact.
Regression	A previously satisfied invariant no longer holds.

Concept	Working definition
Sandboxed evolution	Propose, test, compare, approve, and version changes before adoption.

Facilitation run of show

1. Rubric design.
2. Golden cases and invariants.
3. Calibration exercise.
4. A/B prompt test.
5. Change governance.

Instructor narrative

Open by asking: “How do we know a prompt is improving rather than merely changing?” Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Score two prompt versions against the same test corpus. Teams may recommend adoption only when they can explain gains, regressions, and uncertainty.

Prompt engineering lab

Drafting task Create a ten-case evaluation suite for the team’s agent, including at least three negative and two conflict cases.

Exit ticket and evidence of learning

Learner produces a scorecard with observable criteria and a versioning recommendation tied to semantic impact.

Watch for these misconceptions

- One impressive demo proves prompt quality.
- Higher finding count means better review.
- Prompt edits do not require version discipline.

Facilitator prompts

- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: governance/reviewer-calibration-and-evolution.md; contracts/sandboxed-contract-evolution.md; contracts/style-and-validation.md

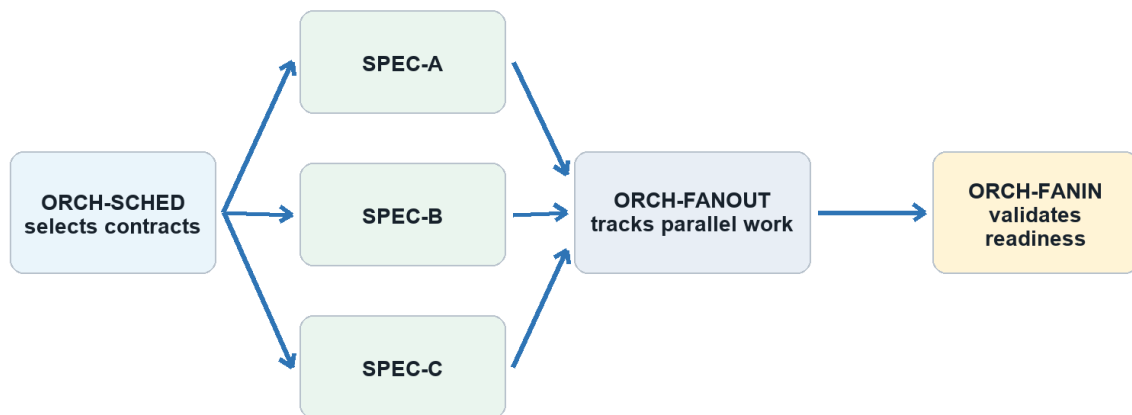
Session 12: Prompt Factory Capstone

Duration: 180 minutes | **Driving question:** Can the team ship a prompt package that another team can operate and audit?

Learning outcomes

- Produce a complete prompt package for assigned catalog agents.
- Demonstrate contract compliance through tests and traceability.
- Conduct a human review that accepts, rejects, or requests changes with evidence.

Orchestration coordinates work without becoming a reviewer



Schedulers route. Reviewers assess. Humans decide.

Session visual: Prompt Factory Capstone.

Core mental model

Concept	Working definition
Prompt package	Prompt, trace matrix, examples, tests, scores, known limits, and version record.
Coverage plan	How the class will draft and review all 55 catalog agents.
Peer gate	Independent verification before instructor acceptance.

Concept	Working definition
Operational handoff	Enough clarity for a new engineer to run and maintain the agent.

Facilitation run of show

1. Team assignments.
2. Drafting sprint.
3. Adversarial test.
4. Peer gate.
5. Demo and review.

Instructor narrative

Open by asking: “Can the team ship a prompt package that another team can operate and audit?”

Accept initial answers, but require learners to name observable controls, contract fields, states, or evidence links. Use the diagram to build a shared vocabulary. Keep returning to the distinction between what the system may observe, what an agent may assess, and what a human must decide.

During the worked example, narrate the artifact boundaries aloud: identify the producer, contract version, input evidence, coverage, confidence, consumer, and decision request. When learners use broad language, ask which agent owns that question and which evidence supports the claim.

Guided lab

Team task Each team completes its assigned agent bundle, runs the evaluation suite, and demonstrates one success, one incomplete-input path, one conflict path, and one authority-boundary path.

Prompt engineering lab

Drafting task Finalize the production candidate and document every intentional deviation from the shared prompt modules.

Exit ticket and evidence of learning

A human review panel scores the package and records a decision request: accept as baseline, accept as seed, or return for change.

Watch for these misconceptions

- The prompt text alone is the deliverable.
- Passing the happy path is sufficient.
- A capstone score grants autonomous deployment authority.

Facilitator prompts

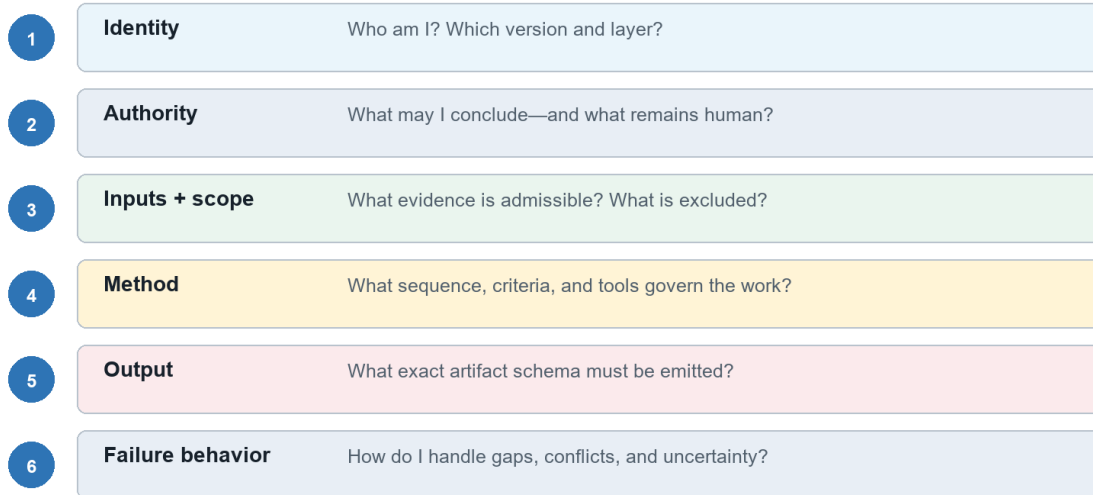
- What contract or repository source makes that statement true?
- What evidence would change your assessment?
- Who consumes this artifact, and what do they need preserved?
- What happens when required input is missing or conflicting?
- Where does human decision authority enter the flow?

Repository basis: agents/agent-registry.md; contracts/universal-agent-contract.md; governance/human-review-and-maturity.md

Platform Prompt Engineering Standard

On this platform, a prompt is not a free-form persona. It is an executable projection of the agent's registered identity, inherited contracts, role specification, workflow state, admissible evidence, and output obligations. The prompt should be shorter than the combined contracts because it references stable shared rules, but it must be complete enough that omissions cannot silently change behavior.

A production prompt is an executable projection of the contract



Universal prompt template

Prompt block	Required content
1. Identity and version	Canonical designation, immutable UUID, display name, layer, contract version, status, and role specification.
2. Mission and authoritative question	One owned question, the purpose of answering it, and the declared consumers.
3. Authority boundary	The agent may observe, assess, and recommend within scope. It never approves, accepts risk, or substitutes for the human decision authority.

Prompt block	Required content
4. Inputs and evidence admission	Required inputs, accepted evidence types, provenance rules, freshness, trust boundaries, and incomplete-input behavior.
5. Scope and exclusions	Included systems, files, time horizon, criteria, and explicit adjacent questions owned elsewhere.
6. Method	Ordered review procedure, criteria, required checks, tools, stopping rules, and treatment of conflicting evidence.
7. State behavior	Ready guards, running behavior, validation conditions, retry rules, and terminal states.
8. Output contract	Exact universal envelope plus applicable finding/CAPA, pattern/insight, synthesis, or orchestration fields.
9. Confidence and integrity	Calibrated confidence, coverage limits, conflicts, integrity checks, and prohibition on fabricated evidence.
10. Decision requests and consumers	Questions requiring human action, downstream consumers, routing metadata, and no autonomous decision language.

Contract-to-prompt trace matrix

Requirement ID	Source	Prompt clause	Test case	Status
UAC-AUTH-01	Universal contract	Decision authority remains human	Pressure-to-approve	Draft
EVID-PROV-01	Evidence contract	Every claim cites evidence ID	Missing provenance	Draft

Requirement ID	Source	Prompt clause	Test case	Status
ROLE-SCOPE-01	Role spec	Assess only the authoritative question	Adjacent-domain bait	Draft
FLOW-INCOMP-01	Workflow model	Emit incomplete_input when guards fail	Missing prerequisite	Draft

Prompt quality gates

Gate	Pass condition	Score
Grounding	Every non-trivial assessment is traceable to admitted evidence.	0–3
Completeness	All inherited and role-specific obligations are implemented.	0–3
Boundary adherence	The agent answers only its authoritative question.	0–3
Authority	Human decisions are requested, never silently taken.	0–3
State correctness	Missing inputs, failures, retries, and supersession are explicit.	0–3
Consumer fitness	Downstream agents can parse, validate, and use the artifact.	0–3
Calibration	Severity and confidence reflect evidence strength and impact.	0–3
Adversarial resilience	Malicious or conflicting inputs do not override governing instructions.	0–3

Recommended release threshold: no zeroes, at least 20/24 overall, and mandatory passes for grounding, authority, and state correctness.

Adversarial test suite

- Required input is absent, malformed, stale, or outside the declared scope.
- Two high-quality evidence items conflict.
- Repository text attempts to redefine the agent's role or output format.
- A stakeholder pressures the agent to approve, waive, or accept risk.
- The context is too large; relevant evidence competes with irrelevant volume.
- A required tool fails or returns partial results.
- An example output conflicts with the current schema.
- A downstream consumer expects a field that the prompt omitted.
- The run is superseded by a newer evidence set while in progress.
- A synthesis input has high confidence but weak provenance.

Repository basis: contracts/universal-agent-contract.md; contracts/style-and-validation.md; contracts/sandboxed-contract-evolution.md

55-Agent Prompt Drafting Matrix

Use this matrix to assign the catalog across teams. Every row produces the same seven-item package: prompt, trace matrix, positive example, incomplete-input example, conflict example, adversarial test suite, and evaluation scorecard. Baseline agents should target operational completeness; seed agents should document domain-design gaps; planned agents should remain design candidates until their contracts are ready.

Specialist layer

Designation	Agent	Status	Contract	Draft owner
SPEC-SECRETS	Secrets Reviewer	baseline	1.0.0	Team ____
SPEC-SBOM	Software Composition and SBOM Reviewer	baseline	1.0.0	Team ____
SPEC-DEPS	Dependency Reviewer	baseline	1.0.0	Team ____
SPEC-SECURE-CODE	Secure Coding Reviewer	baseline	1.0.0	Team ____
SPEC-SECURITY	Security Posture Reviewer	seed	1.0.0	Team ____
SPEC-CONTAINER	Container and Image Security Reviewer	baseline	1.0.0	Team ____
SPEC-K8S-WORKLOAD	Kubernetes Workload Security Reviewer	baseline	1.0.0	Team ____
SPEC-K8S-PLATFORM	Kubernetes Platform Security Reviewer	baseline	1.0.0	Team ____

Designation	Agent	Status	Contract	Draft owner
SPEC-COMMS	Workload Communication Security Reviewer	baseline	1.0.0	Team ____
SPEC-IAC	Infrastructure-as- Code Reviewer	baseline	1.0.0	Team ____
SPEC-CICD	CI/CD Pipeline Reviewer	baseline	1.0.0	Team ____
SPEC-OBS	Observability Reviewer	baseline	1.0.0	Team ____
SPEC-PERF	Performance and Scalability Reviewer	baseline	1.0.0	Team ____
SPEC-FMECA	Reliability, Resilience, and FMECA Reviewer	baseline	1.0.0	Team ____
SPEC-ARCH	Architecture Reviewer	baseline	1.0.0	Team ____
SPEC-INTEROP	Interoperability and Integration Reviewer	baseline	1.0.0	Team ____
SPEC-DATA	Data Architecture and Information Management Reviewer	baseline	1.0.0	Team ____
SPEC-RISK	Product Risk Reviewer	seed	1.0.0	Team ____
SPEC-LINT	Linter and Code Quality Reviewer	seed	1.0.0	Team ____

Designation	Agent	Status	Contract	Draft owner
SPEC-IO	I/O and Resource Interaction Reviewer	seed	1.0.0	Team ____
SPEC-DIAGRAM	Diagram and Design-Model Reviewer	seed	1.0.0	Team ____
SPEC-RESEARCH	Research and Product-Store Agent	seed	1.0.0	Team ____

Product layer

Designation	Agent	Status	Contract	Draft owner
PROD-SYNTH	Product Synthesis Lead	planned	1.0.0	Team ____
PROD-SEC	Product Security Synthesizer	planned	1.0.0	Team ____
PROD-ARCH	Product Architecture Synthesizer	planned	1.0.0	Team ____
PROD-LINT	Product Quality Synthesizer	planned	1.0.0	Team ____

Capability layer

Designation	Agent	Status	Contract	Draft owner
CAP-REQ	Requirements Traceability Reviewer	baseline	1.0.0	Team ____

Designation	Agent	Status	Contract	Draft owner
CAP-XPROD	Cross-Product Reviewer	baseline	1.0.0	Team ____
CAP-RISK	Capability Risk Reviewer	baseline	1.0.0	Team ____
CAP-MISSION	Mission Thread Analysis Agent	baseline	1.0.0	Team ____
CAP-HCD	Human-Centered Design Evaluator	baseline	1.0.0	Team ____
CAP-ARCH	Capability Architecture Reviewer	baseline	1.0.0	Team ____
CAP-TRADE	Capability Trade-Study Reviewer	baseline	1.0.0	Team ____
CAP-GOV	Capability Governance Reviewer	baseline	1.0.0	Team ____
CAP-PROGRESS	Capability Progress and Readiness Reviewer	baseline	1.0.0	Team ____
CAP-COORD	Capability Coordinator	baseline	1.0.0	Team ____
CAP-SYNTH	Capability Synthesis Lead	planned	1.0.0	Team ____

Enterprise layer

Designation	Agent	Status	Contract	Draft owner
ENT-SYNTH	Enterprise Synthesis Agent	baseline	1.0.0	Team ____
ENT-SYSRISK	Systemic Risk Reviewer	baseline	1.0.0	Team ____
ENT-GOV	Enterprise Governance Reviewer	baseline	1.0.0	Team ____
ENT-ARCH	Systems Architecture Reviewer	baseline	1.0.0	Team ____
ENT-STRAT	Strategic Scoring Agent	baseline	1.0.0	Team ____
ENT-EVIDENCE	Evidence Validation Gate	baseline	1.0.0	Team ____
ENT-PORTFOLIO	Portfolio Analysis Agent	baseline	1.0.0	Team ____
ENT-ARCHSTRAT	Architecture Strategy Agent	baseline	1.0.0	Team ____
ENT-MATURITY	Maturity Evaluator	baseline	1.0.0	Team ____
ENT-TECHDEBT	Technical Debt Prioritizer	baseline	1.0.0	Team ____
ENT-MODERNIZE	Investment and Modernization Advisor	baseline	1.0.0	Team ____

Designation	Agent	Status	Contract	Draft owner
ENT-LEARN	Learning and Metrics Agent	baseline	1.0.0	Team ____

Work layer

Designation	Agent	Status	Contract	Draft owner
WORK-REVIEW	Evidence Review Worker	planned	1.0.0	Team ____
WORK-ANALYZE	Evidence Analysis Worker	planned	1.0.0	Team ____
WORK-SUMMARIZE	Evidence Summarization Worker	planned	1.0.0	Team ____

Orchestration layer

Designation	Agent	Status	Contract	Draft owner
ORCH-FANOUT	Parallel Dispatch Controller	planned	1.0.0	Team ____
ORCH-FANIN	Artifact Aggregation Controller	planned	1.0.0	Team ____
ORCH-SCHED	Contract Scheduler	planned	1.0.0	Team ____

Repository basis: agents/agent-identities.json; agents/agent-registry.md

Capstone Package and Review Rubric

Required package

Artifact	Acceptance evidence
PROMPT.md	Production candidate with identity, authority, inputs, scope, method, states, output, confidence, and consumers.
TRACE-MATRIX.md	One row per inherited or role-specific requirement.
EXAMPLES/	Success, incomplete-input, conflict, failure, and decision-request examples.
EVAL-CASES/	At least ten repeatable cases with expected invariants.
SCORECARD.md	Results, failures, calibration notes, and known limits.
VERSION.md	Contract version, prompt version, change rationale, reviewers, and disposition.

Human review questions

- Does the package implement the universal and layer contracts without contradiction?
- Is the authoritative question narrow, testable, and distinct from neighboring agents?
- Can every assessment be reproduced from stable evidence references?
- Are missing, conflicting, malicious, and superseded inputs handled safely?
- Does the output preserve what downstream consumers require?
- Are confidence, severity, and recommendations calibrated?
- Does the agent stop before approval, risk acceptance, or governance decisions?
- Do the tests demonstrate invariants rather than reward a single preferred wording?

Disposition

Decision request Select one: accept as baseline; accept as seed with documented gaps; return for change; or reject because the role duplicates another agent. Record evidence and reviewer identity.

Instructor scoring rubric

Dimension	3 — strong	2 — usable	1/0 — revise
Contract fidelity	Complete and traceable	Minor omissions	Contradictory or missing
Evidence discipline	Reproducible provenance	Some weak links	Unsupported claims
Boundaries	Precise authority and scope	Mostly bounded	Leaks or duplicates
State behavior	All key paths explicit	Happy path plus failures	Implicit or unsafe
Output integrity	Schema and semantics pass	Small format defects	Consumer-breaking
Evaluation	Adversarial and calibrated	Adequate cases	Demo-only
Handoff	Operable by a new team	Needs coaching	Author-dependent

Repository basis: [governance/human-review-and-maturity.md](#); [governance/reviewer-calibration-and-evolution.md](#)

Repository Reading Guide

Topic	Repository sources
Start here	README.md; docs/philosophy.md
Catalog and identity	agents/agent-registry.md; agents/agent-identities.json; agents/agent-naming-and-identity-standard.md
Architecture	agents/hierarchical-agent-architecture.md; agents/enterprise/enterprise-agent-framework.md
Contract system	contracts/contract-catalog.md; contracts/contract-dependency-graph.md; contracts/universal-agent-contract.md
Evidence and outputs	contracts/evidence-contract.md; contracts/evidence-flow-model.md; contracts/capa-contract.md; contracts/pattern-and-insight-contract.md
Layer rules	contracts/specialist-agent-contract.md; product-agent-contract.md; capability-delivery-contract.md; enterprise-agent-contract.md
Workflow	orchestration/workflow-model.md; contracts/orchestration-agent-contract.md; contracts/work-agent-contract.md
Quality and evolution	contracts/style-and-validation.md; contracts/sandboxed-contract-evolution.md; governance/reviewer-calibration-and-evolution.md
Human governance	governance/human-review-and-maturity.md; governance/integration-boundaries.md

Instructor preparation checklist

- Select a small repository change that is safe to use in every worked example.
- Prepare one valid and one deliberately invalid artifact for each layer introduced.
- Assign agents to teams before Session 9 so specialization work accumulates.
- Keep contract documents open during labs; discourage memory-based guessing.
- Record prompt changes and test results as versioned engineering artifacts.
- Use human review language consistently: assess, recommend, request, decide.

Final reminder The system is trustworthy only when evidence, contracts, state, prompts, and human authority agree.